BIG DATA · GROUP ASSIGNMENT 2 · TEAM SNUSIK

An Airbnb price lakehouse for US Northeast — and what happened when a third city arrived

Boston (A) and Jersey City (B) to build it · New York City (C) as the new client

$73.6

test MAE on Boston + Jersey City (baseline $124.7)

$143.2

first run on New York, model unchanged (baseline $164.2)

$114.7

New York after our changes — still trained on A + B only

Data: Inside Airbnb (CC BY 4.0), snapshots 2026-09-14 … 2026-09-21

THE TASK

Build pricing for two clients, then find out whether it is a pipeline or a prototype

The story

- A short-term-rental analytics start-up with clients in **Boston** and **Jersey City**.
- For them: a lakehouse over Airbnb data, a dashboard, a Genie agent and a model that suggests a nightly price.
- A week later sales signs **New York City**: “the pipeline takes the city as a parameter, right?”
- We ran it unchanged, measured what broke, fixed it, and report the gap honestly.

What we built

- **EDA notebook** on A and B with a leakage review.
- **Medallion lakehouse** on Databricks: one job, `city` is a parameter.
- **Price model** (scikit-learn) vs a median baseline, honest test split.
- **AI/BI dashboard** (3 pages) and a **Genie space**, both as code.
- **City C**: first run, 4 pipeline fixes, a new model version, the gap explained.

Everything is in the repository as code: bundle, notebooks, dashboard JSON, Genie config.

DATA

One row = one listing at scrape time; a third of New York has no price

CITY	ROLE	SNAPSHOT	LISTINGS	NO PRICE	MEDIAN PRICE
Boston	A · training	2026-09-19	4,368	20.2%	$253
Jersey City	B · training	2026-09-21	2,330	10.1%	$215
New York City	C · new client	2026-09-14	29,918	35.1%	$181

Detailed `listings.csv.gz`, 90 columns, identical schema in all three cities. 14 columns are 100% empty in this scrape (all host response fields, `bathrooms`, `instant_bookable`).

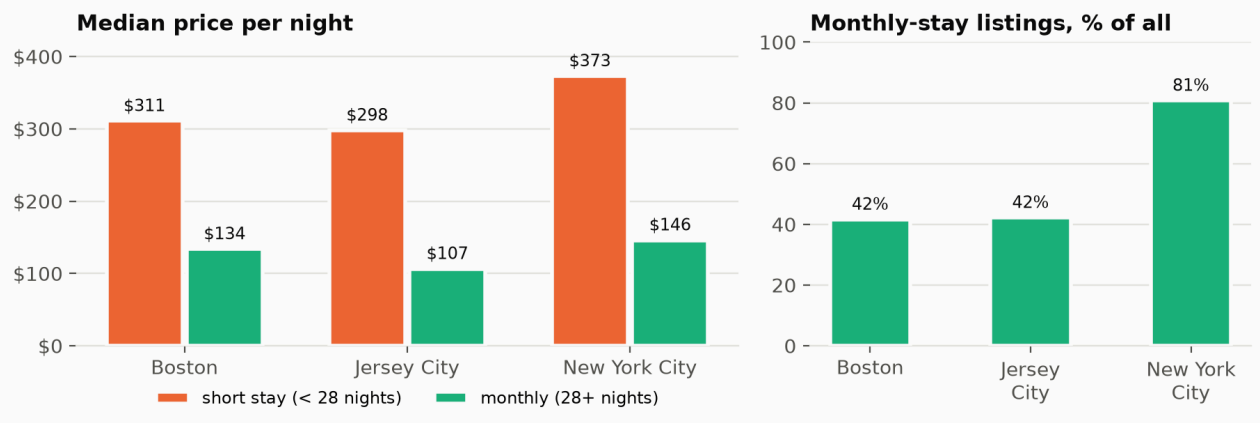
Why New York as city C

The region's largest and most expensive market, and the only one under Local Law 18 (registration needed for stays under 30 nights). The hardest realistic test of transfer.

City C was not loaded until the pipeline and model for A and B were committed.

EXPLORATORY ANALYSIS (A AND B)

Stay length matters as much as size: monthly rentals advertise less than half the nightly price



What we learned

- **Missing prices are not random.** 65% of records carried over from the previous scrape and 92–100% of fully blocked calendars have no price. We model the active, bookable listings.
- **Review fields are structurally empty** exactly when a listing has no reviews.
- **Outliers, two kinds:** 7 implausible “parking” prices (≥ $750 per guest per night, e.g. a 2-bedroom at $11,680) are excluded; 47 large luxury homes are kept.
- **Right-skewed prices** → model log price.

New York is on the chart for context; it was profiled only after week 1.

LEAKAGE REVIEW

Six columns are the price in disguise — they match it for 100% of priced listings

COLUMN	WHY IT LEAKS
`price_quote_price_per_night`	equals `price` (to rounding) for every priced listing
`price_quote_total_price`	price × nights (+ taxes)
`price_quote_raw`	JSON quote: “3 nights x $189.00”
`price_quote_checkin/checkout_date`	part of the same quote; null exactly when price is null
`estimated_revenue_l365d`	= price × estimated occupancy

Also excluded

- **Booking outcomes** — availability, occupancy, recent review counts: they react to the price (soft leaks) and a new listing would not have them.
- **Free text** — 1.7% of descriptions quote a dollar amount; we use no text features.
- **Identifiers** and the 14 empty columns.

Split before anything else

20% of listings per city go to test by `hash(id) mod 100 < 20`, fixed in gold before any model existed.

ARCHITECTURE

A medallion lakehouse where every layer has one job and every table holds all cities

Source

Inside Airbnb `listings.csv.gz` one pinned snapshot per city in `cities.json`

→

Bronze · raw

UC volume with the file as downloaded `listings_raw`: every column a STRING + city, snapshot, source file, ingest time

→

Silver · clean

`listings`: typed with `try_cast`, parsed price / bathrooms / amenities, de-duplicated, price class and flags, leak columns dropped, data-quality gates

→

Gold · consume

`listings` · `city_metrics` · `neighbourhood_metrics` `listing_features` (+ fixed split) `price_predictions` · `model_metrics` `pipeline_runs` (audit)

→

Serve

Model versions in a UC volume (+ MLflow)
AI/BI dashboard
Genie space

Strings in bronze. Typing at read time would silently null bad values; silver counts every value that fails to parse and stops the run above 1%.

One table per layer, partitioned by city. Re-running a city rewrites only its partition; cross-city comparison is a `GROUP BY`.

Immutable model versions. The first New York run had to use the week-1 model exactly, so `v1` is never overwritten.

PIPELINE

One parameterised pipeline: adding a city is one line of configuration, not a copy of code

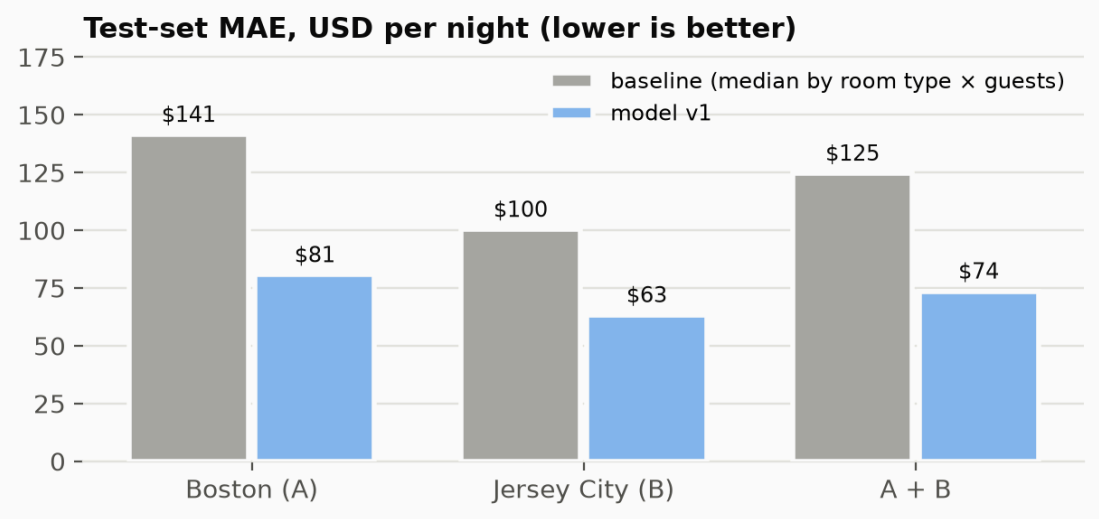
JOB (DATABRICKS ASSET BUNDLE)	WHAT IT DOES
`airbnb_city_pipeline` param `city`	setup → bronze → silver → gold
`airbnb_etl`	`for_each` city in a list → the job above
`airbnb_train_price_model`	ETL for training cities → train, evaluate, save version
`airbnb_onboard_city`	city pipeline → score with a saved version, no refit

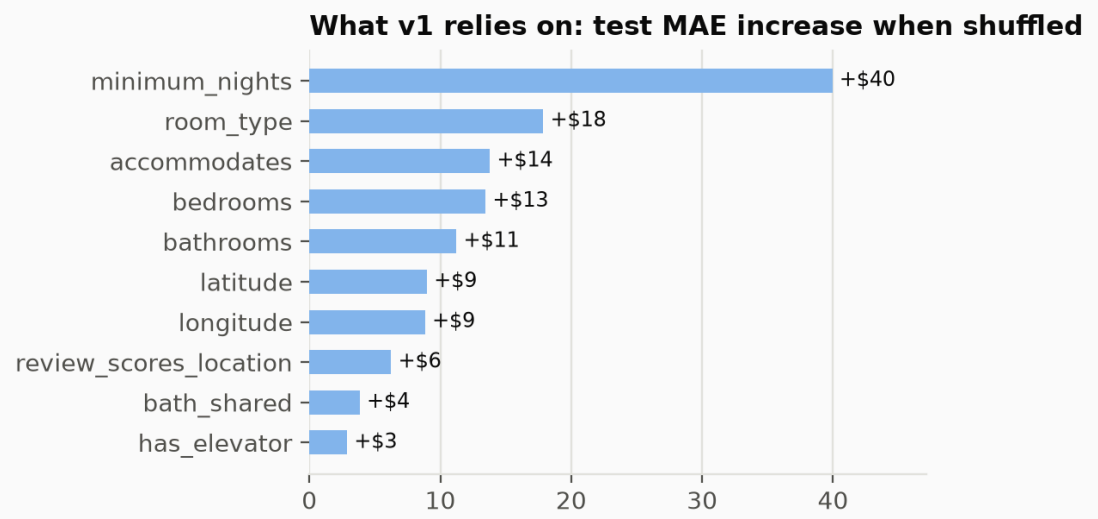
Running it

- Serverless notebooks, Spark SQL with ANSI `try_cast`; shared code in `src/airbnb`.
- Every step appends row counts and checks to `gold.pipeline_runs`.
- **Free Edition hit its limits** during the project (compute quota, then the workspace became inactive). `scripts/run_local.py` runs the *same notebooks* on local Spark 4 + Delta; the numbers in this deck come from those runs. Once the workspace was back, every job ran on Databricks and reproduced them within ~1% (NYC: v1 $144.19, v2 $115.08).

PRICE MODEL (V1, END OF WEEK 1)

Gradient boosting on log price beats the median baseline by 41% on held-out listings





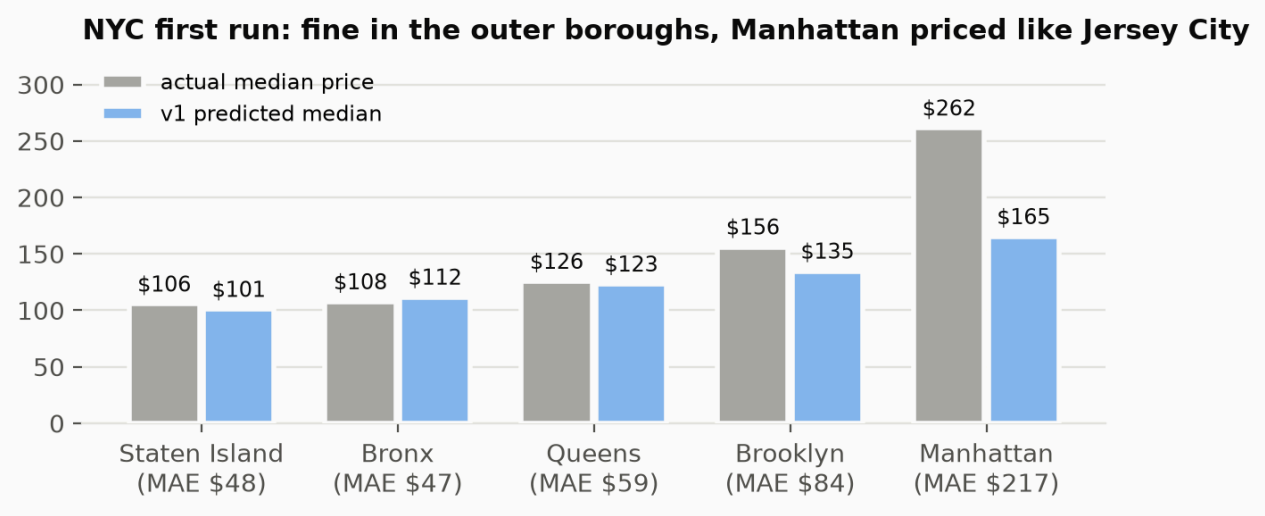
Baseline: median training price for the same room type and number of guests. **Model:** chosen by 5-fold CV on the training split among ridge and three boosting variants (CV MAE $70.7). Absolute loss on log price = predicts the median price, the MAE-optimal estimate. **Features:** unit size and privacy, stay rules, location, amenities, host, reviews — no leak columns.

CITY C · FIRST RUN

Everything ran green — and the model priced Manhattan like Jersey City

STEP	RESULT
Download, bronze	29,918 rows, same 90 columns
Silver	0 parse failures, 0 duplicates
Gold, scoring	all listings scored with `v1`
MAE, NYC test	**$143.2 (baseline $164.2)**
MAE, A + B test	$73.6
Mean error	–$128 a night (under-pricing)

Nothing failed and **nothing warned** — although 35% of the target was missing and 81% of listings were monthly rentals.



CITY C · WHY

The model's sense of “bigger is pricier” transferred; its sense of where and how much did not

- **A different market.** Under Local Law 18, 81% of NYC listings require 30+ nights (Boston, Jersey City: 42%). The few short stays are pricey: median $373 vs $311 in Boston. They are 26% of priced listings but **49% of the error**.
- **A price level never seen.** Error grows with price: listings above $700 are predicted at a median of $288. Hotel rooms: $629 actual, $280 predicted.
- **Location features meant nothing — or something else.** `city` 100% unseen, neighbourhoods 99% unseen. “Chinatown” exists in both cities: Manhattan's Chinatown got Boston's Chinatown effect ($405 actual, $220 predicted). NYC coordinates fall next to Jersey City's.
- **Mostly a level problem.** Rescaling every v1 prediction by one NYC factor (×1.29) would already cut MAE from $138 to $122 on all priced listings.

CITY C · WHAT WE CHANGED

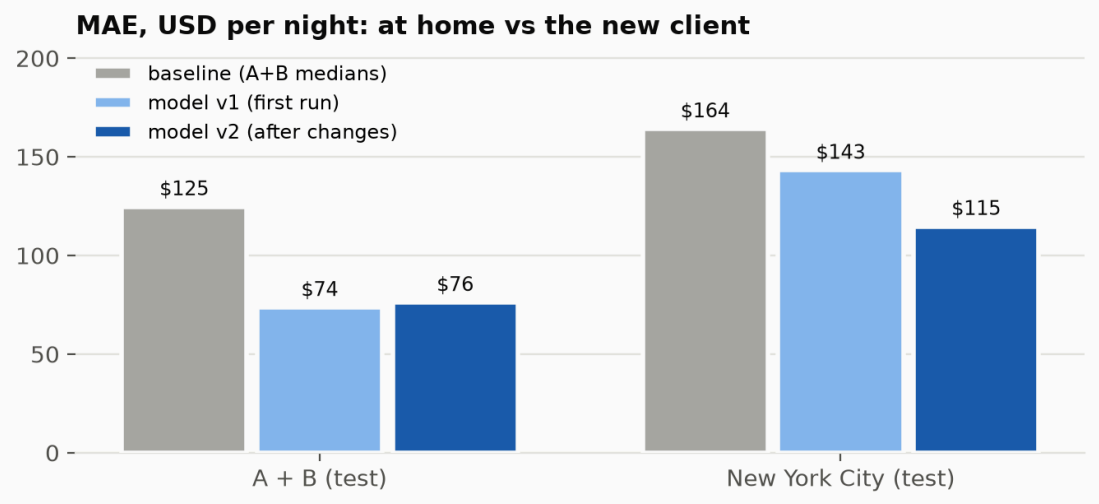
Four things the pipeline should have done from day one, three things the model could not know

KIND	CHANGE	REASON
config	add `new-york-city` to `cities.json`	the designed way to add a city
pipeline	carry `neighbourhood_group` (borough) to silver and gold	empty in A/B, full in C — silently dropped
pipeline	required vs optional source columns	city files differ; a missing optional column must not crash silver
pipeline	target and market profile with warnings	first run was green with 35% of the target missing; now NYC raises 4 warnings
pipeline	input-drift report at scoring	nobody was told the model scored unseen inputs (`city` 100%, neighbourhood 99%)
model	drop `city`, `neighbourhood`, latitude, longitude	city-specific: unseen levels, homonyms, coordinates out of context
model	market comps: median price of the 20 nearest same-type listings (and 50 of any type) from the city's own training split — never the listing itself or its host	a price level the model never saw: anchor on the local market
model	predict `log(price / comp price)` instead of `log(price)`	trees cannot extrapolate a level; now they only learn adjustments

Checked and kept: the $750-per-guest rule (NYC's 111 flags are placeholders and $1,000+ single rooms), the 28-night threshold, the baseline.

CITY C · THE NUMBERS

v2 closes 41% of the New York gap without a single New York training example

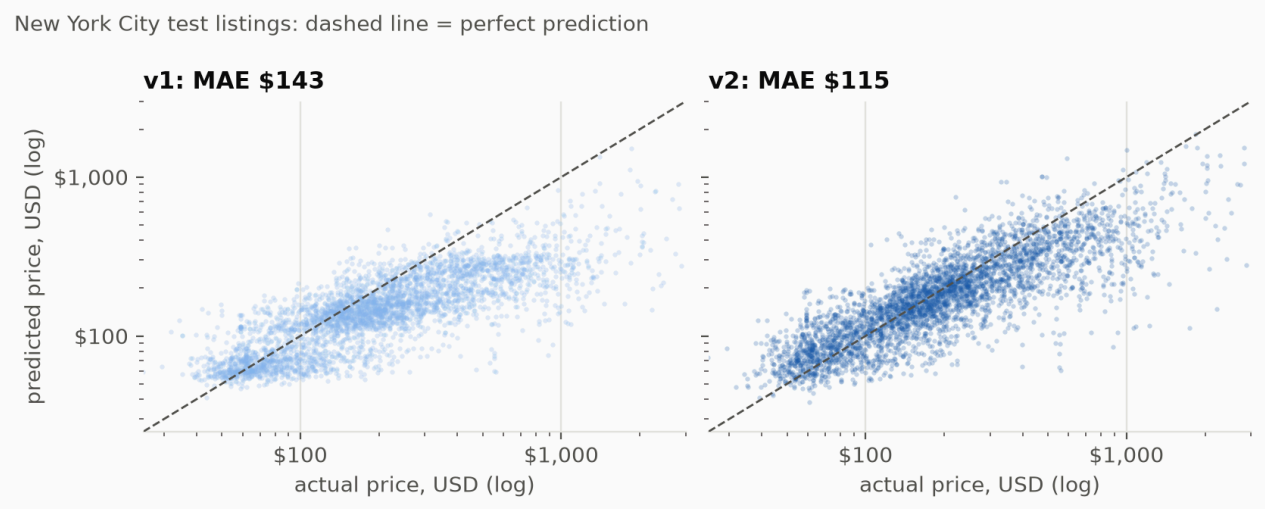


THE $69.5 FIRST-RUN GAP	$ / NIGHT
v2 gives up at home (A+B $73.6 → $76.1)	2.5
features and price level that did not transfer (v1 → v2)	**28.5**
learnable only from NYC data (v2 → v2 retrained with NYC: $87.6)	27.1
NYC is a more spread-out market ($87.6 vs $76.1)	11.5

In relative terms the NYC-trained reference is almost as good as at home (MAPE 25.4% vs 22.8%); the same relative miss on a $400–$7,800 Manhattan listing is many more dollars.

NYC test split = the same 20% hash split, never trained on by v1 or v2

Day one: outer boroughs and monthly rentals are priced like at home; Manhattan short stays need NYC data



- v2 on Queens, Bronx, Staten Island: **$43–63 MAE**, close to Boston and Jersey City.
- Manhattan: $231 → $175 MAE; short stays still predicted $301 vs $380 actual.
- Retraining with the NYC snapshot (the `v2_abc` reference) reaches **$87.6** — what we would ship after the first week.
- The scatter shows v1's ceiling: predictions flatten out around $300 whatever the actual price; v2 follows the diagonal much further.

Every dashboard page states its finding; Genie answers in medians, per city, on test listings only

AI/BI dashboard — 3 pages, 12 datasets

- **City overview:** “Boston has the priciest typical night ($253); New York is a monthly-rental market (81%)”. KPIs, room-type mix, top-20 neighbourhoods.
- **Cross-city comparison:** “An entire home costs the same in both cities (~$300); Jersey City is cheaper only for private rooms (–32%) and small units”.
- **Model:** baseline vs v1 vs v2 at home and in New York, predicted vs actual, error by room type and area, most mis-priced listings.

Generated from `dashboards/build_dashboard.py`, deployed by the bundle.

Genie tricks

- Vocabulary: “City A/B/C” → keys; “price” means **median**; “entire home” → room type.
- One boolean for every price statistic: `has_valid_price`.
- Sign convention written twice: error = predicted – actual, negative = under-priced.
- Evaluate on `split = 'test'`; “first run” = v1, current = v2.
- Unity Catalog comments on gold columns, join specs with relationship types, 7 example SQLs.

“Median price of entire homes in City A?” → **$297.16**. “Where does the model under-price most?” → **Manhattan** (–$125 mean error with v2).

WRAP-UP

Challenges, and who did what

Challenges

- **Free Edition limits:** compute quota exhausted, then the workspace turned inactive mid-project. We kept the code Databricks-native and ran the same notebooks locally on Spark 4 + Delta.
- **A moving source schema:** new `price_quote_*` columns (all leaks), 14 empty host columns.
- **Spark ANSI mode:** `CAST('1.0' AS INT)` fails (Jersey City writes `1.0`); we cast through DOUBLE with `try_cast` and count every value that stops parsing.
- **The honest split:** comps had to exclude the listing itself *and* its host's other units, or the test score would flatter us.

Team contributions

MEMBER	CONTRIBUTION
Yuliana Hrynda	All parts, together: EDA and leakage review, lakehouse and pipeline, price model, city C analysis, dashboard, Genie, presentation
Ostap Mnykh	
Mykhailo Brytan	
Yulian Zaiats	

We worked on every part as a team rather than splitting it by person.